docker container create --name webapp -p 8081:80 httpd

FROM ubuntu:20.04

MAINTAINER ADMIN "admin@cloudthat.com"

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && \

apt-get -q -y install apache2 \

php7.4 \

php7.4-fpm \

php7.4-mysql \

libapache2-mod-php7.4

ADD http://wordpress.org/latest.tar.gz /tmp

RUN tar xzvf /tmp/latest.tar.gz -C /tmp \

&& cp -R /tmp/wordpress/*/var/www/html

RUN rm /var/www/html/index.html && \

chown -R www-data:www-data /var/www/html

EXPOSE 80

CMD ["/usr/sbin/apache2ctl","-D","FOREGROUND"]

apiVersion: v1

kind: Pod

metadata:

name: nginx

spec:

containers:

- name: con1

image: nginx:1.14.2

ports:

- containerPort: 80

- name: con2

image: redis

ports:

- containerPort: 90

apiVersion: apps/v1
kind: Deployment
metadata:
name: nginx-dep
labels:
app: nginx-dep
spec:
replicas: 3
strategy:
type: Recreate
selector:
matchLabels:
app: nginx-app
template:
metadata:
labels:
app: nginx-app
spec:
containers:
- name: nginx-ctr
image: nginx:1.12.2
ports:
- containerPort: 80

apiVersion: apps/v1
kind: DaemonSet
metadata:
name: fluent-ds
labels:
app: fluent-ds
spec:
selector:
matchLabels:
app: fluentd-app
template:
metadata:
labels:
app: fluentd-app
spec:
containers:
- name: fluentd-ctr
image: nginx

MYSQL_USER=admin

MYSQL_PASSWORD=password MYSQL_ROOT_PASSWORD=password

kind: PersistentVolume
apiVersion: v1
metadata:
name: pv-volume
spec:
storageClassName: manual
capacity:
storage: 1Gi
accessModes:
- ReadWriteMany
hostPath:
path: "/pvdir"

kind: PersistentVolumeClaim
apiVersion: v1
metadata:
name: pv-claim
spec:
storageClassName: manual
accessModes:
- ReadWriteMany

resources:

requests:
storage: 1Gi

kind: Pod
apiVersion: v1
metadata:
name: pv-pod
spec:
volumes:
- name: pv-storage
persistentVolumeClaim:
claimName: pv-claim
containers:
- name: pv-container
image: nginx
ports:
- containerPort: 80
name: "http-server"
volumeMounts:
- mountPath: "/usr/share/nginx/html"
name: pv-storage
nodeSelector:
kubernetes.io/hostname: ip-172-20-33-138.ap-south-1.compute.internal

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
name: hello-ingress
annotations:
nginx.ingress.kubernetes.io/rewrite-target: /
kubernetes.io/ingress.global-static-ip-name: "global-ingress"
spec:
rules:
- http:
paths:
- path: /
pathType: ImplementationSpecific
backend:
service:
name: nginx-svc
port:
number: 80
- path: /httpd
pathType: ImplementationSpecific
backend:
service:
name: httpd-svc
port:
number: 80

apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
namespace: default
name: pod-reader
rules:
- apiGroups: [""] # "" indicates the core API group
resources: ["pods"]
verbs: ["get", "watch", "list"]

apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
name: read-pods
namespace: default
subjects:
- kind: User
name: Ramesh
apiGroup: rbac.authorization.k8s.io
roleRef:
kind: Role
name: pod-reader
apiGroup: rbac.authorization.k8s.io

Deployment of sample microservices application in
Kubernetes
Installation and configuration of service mesh
Traffic routing and control mechanisms
Observability and tracing of service-to-service
communication
Enabling, disabling, and modifying traffic rules dynamically

Observability and monitoring of Cluster/GKE

on Google cloud